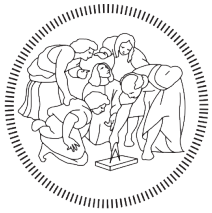
Artificial Neural Networks and Deep Learning - Challenge 2
Time Series Classification

December 2022

Riccardo Luigi Aielli
Salvatore Buono
Simone Buranti



POLITECNICO
MILANO 1863

Contents

1 Introduction **2**

2 Exploratory Data Analysis **2**
2.1 Dataset split 2
2.2 Preprocessing 2
2.2.1 Standardization 2
2.2.2 Feature Reduction 2
2.2.3 Window Sliding 3
2.2.4 OverSampling 3
2.2.5 Data Augmentation 3
2.3 Trying some networks 3
2.3.1 Adding the attention layer 3

3 Ensemble **3**
3.1 Improving the accuracy by combining predictions 4
3.2 Further developments 4

1 Introduction

The goal of this project is to build a neural network model able to classify time series into 12 different classes. In this report, we illustrate the process that brought us to reach a 73% accuracy score on the test set. Our work has been performed on Google Colab, as it offers a convenient setup and free computational units for code execution. Throughout the challenge, we adopted a bottom-up approach, starting from simple models and progressively increasing the pipeline complexity.

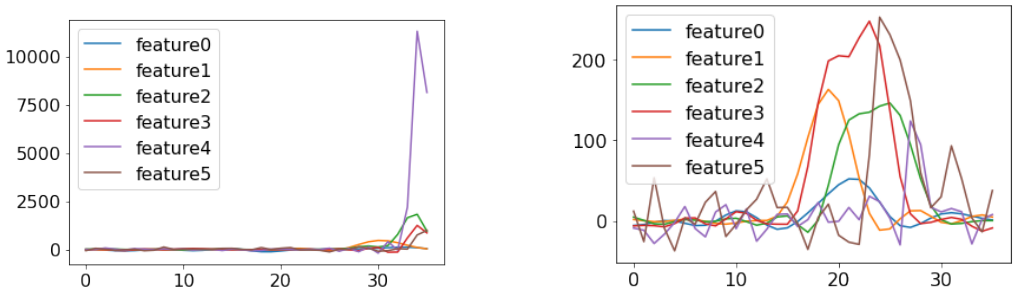
2 Exploratory Data Analysis

2.1 Dataset split

We divided the dataset into training, validation and test sets (80/10/10) using stratified sampling, so that all three sets preserve the class proportions of the original data.

2.2 Preprocessing

During this phase we studied the shape of our data and tried to understand their common properties. Figure 1 shows samples from two different classes.



(a) Sample belonging to class 0

(b) Sample belonging to class 1

Figure 1: Plots of samples belonging to different classes

2.2.1 Standardization

Standardization is a common preprocessing step for many machine learning models, and is typically performed by removing the mean and scaling to unit variance. However, both the mean and the variance are sensitive to outliers, in which case the median and the interquartile range provide more robust statistics. We therefore preprocessed the data with `RobustScaler()`, which removes the median and scales each feature according to its interquartile range.

2.2.2 Feature Reduction

In order to prevent overfitting, we tried to select a subset of features considering the correlation coefficient between each feature and the target variable. Then, we extracted the correlation coefficients for each feature at each time step and finally selected the indices of the features with the highest absolute correlations. However, this approach did not improve our performances, as it decreased the training accuracy without increasing the validation one.

2.2.3 Window Sliding

Window sliding divides each time series into overlapping segments (windows), each of which is used as an input to the classifier. The predictions on the individual windows are then aggregated into a final prediction for the whole series. This technique increases the number of training samples and lets the model focus on local temporal patterns. However, while it improved our local accuracy, in our case it led to overfitting and did not improve performance on the CodaLab test set.

2.2.4 OverSampling

Since the data were unevenly distributed across the 12 classes, we applied oversampling to the minority classes, generating additional samples from the existing ones in order to mitigate class imbalance.

2.2.5 Data Augmentation

We performed data augmentation through two different approaches. Firstly, we implemented by hand a function that modifies training data by adding some noise, scaling, shifting and cropping, but this did not improve our accuracy. Then we used library `tsaug` to augment in a more efficient way, cropping subsequences with length 36, random quantizing to 10, 20, or 30 level sets, with 80% probability, random drifting the signal up to 10% - 50%, with 50% probability, reversing the sequence, and, in the end, concatenating our original data with augmented ones.

2.3 Trying some networks

Starting from exercise sessions, we decided to adapt the networks presented during the lectures (LSTM, BiLSTM, Conv1D) editing both the input shape and the class parameter. Then, we increased the number of hidden layers adjusting their number of units and setting dropout layers to reduce overfitting. Finally, we obtained the best model combining Conv1D with BiLSTM. To compare the different architectures, we evaluated them on our local test set, which, given the held-out test sets available on CodaLab, effectively served as an additional validation set. We also inspected the confusion matrices to identify the classes on which each model performed worse.

2.3.1 Adding the attention layer

Following the advice given in the logbook file, we implemented a type of Attention mechanism between LSTM layers. This can be helpful because it allows the model to focus more on the most relevant time steps and less on the less relevant ones, potentially improving its performance. However, it is computationally intensive to train and implement, since it requires the implementation of a custom layer that computes attention on the positional/temporal dimension. Thus, we decided not to pursue it further.

3 Ensemble

As explained above, our first approach to the challenge was to focus on a single model. However, after several attempts, we noticed that each architecture, when optimized, struggled to obtain an accuracy on the test set higher than 70%. If a single network could not break such a barrier, maybe more networks might succeed instead.

3.1 Improving the accuracy by combining predictions

Ensemble methods are numerous and, according to their implementation, they are suitable for different purposes. What we wanted to achieve was to take advantage of the whole information learnt by our architectures by combining predictions from different feature extraction processes. Thus, we computed for each class the average of the scores assigned to it. For this method to be effective, the involved predictions must rely on intrinsically different architectures. Firstly, we included the best versions of LSTM, BiLSTM, and Conv1D. With this arrangement the method already showed notable improvements on our test set. Despite this, we understood that the ensemble might benefit from a more differentiated set of architectures, as LSTM and BiLSTM differ only in the direction of the timeseries' inspection. Therefore, we implemented a new architecture based on RNN and GRU layers being keen to optimize it, as the advantage brought by a new model might be lost if its performance is considerably poorer than the others.
The inclusion of a fourth model made the ensemble obtained a 4% higher accuracy than single models and clear improvements on precision and recall, with a gain of about 8-10% (Figure 2).

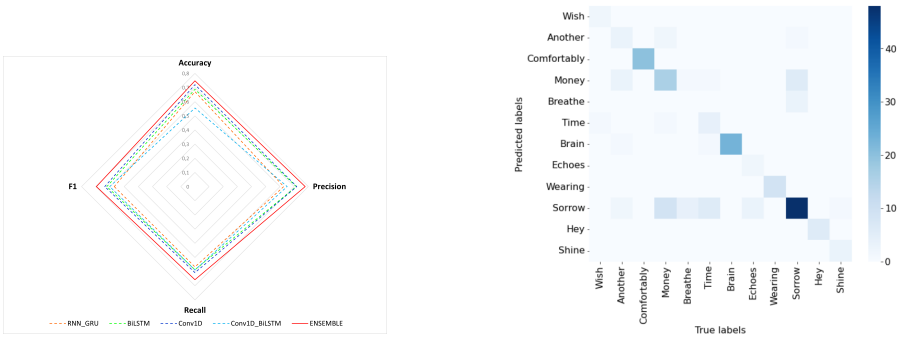


Figure 2: Left: metrics of single models and ensemble. Right: ensemble confusion matrix (local test set).

These local scores were plainly confirmed when we tested the new model on the platform test-set, reaching an overall accuracy of 73.38%.

3.2 Further developments

Having understood the potential of this method we further investigated possible variants. Our goal was to find the model combination which generated a slightly higher class probability for the correct one most of the time. Thus, we adopted the weighted sum combination of predictions. In order to find the best weight vector, we tested all the weight combinations against our local test set. In this phase, we simply included in the group every architecture we had implemented, as counterproductive networks would have been excluded by null weights. However, we were not able to properly exploit this method as, due to the scarcity of test samples, the results were highly split-dependent, and the submitted weighted versions obtained poor results.